



## LLMs - Training LLMs and Generation

Benjamin Piwowarski

Cours 2

<https://master-mind.isir.upmc.fr/llm/slides>

Press ? for help on keyboard shortcuts

[Slides in PDF](#)



Machine Learning &  
Deep Learning for  
Information Access

## TODAY'S LECTURE

1. How to train encoders and decoders?
  - ☞ From pre-training to fine-tuning
2. How to generate text?
  - ☞ Sampling strategies
  - ☞ Maximum likelihood decoding
  - ☞ Guided and constrained generation

## PRACTICAL

1. Generating text with pre-trained models
  - Top-p, top-k, temperature, beam search
2. Fine-tuning an encoder
  - Sentiment analysis on IMDB
3. Guided and Constrained Text Generation
  - Generating answer avoiding tokens
  - Generating formatted (JSON) answers
  - Biasing answers towards positive/negative sentiments

**TRAINING**

## DATA COLLECTION

### UNLABELED TEXT DATA

- 🔑 Web Crawled Data
- 🔑 Books, Articles
- 🔑 Domain-Specific Corpora

### LABELED DATA

- 🔑 Human Annotations (NLP Datasets)

## LOSS FUNCTIONS

### ENCODER

- 🔑 Masked Language Modeling
- 🔑 Next Sentence Prediction
- 🔑 Replaced Token Detection

### DECODER

- 🔑 Maximum Likelihood Estimation
- 🔑 Span Prediction
- 🔑 Reinforcement Learning

# **TRAINING**

## **DATA COLLECTION**

## HIGH-QUALITY, CURATED SOURCES

### 1. Books

  [Smashwords](#)

  [Project Gutenberg](#)

Good for long-range dependencies

### 2. Wikipedia

High-quality encyclopedic content

### 3. News Articles

Quality journalism from major publishers (but (c) issues)

### 4. Forums and Social Media ( Reddit, StackExchange, BlueSky etc.)

Good for dialogues pre-training

# RAW AND FILTERED WEB-SCALE CORPORA

## 1. Common Crawl

Open repository of petabyte-scale web crawl data

 Raw, unfiltered web content

## 2. C4 (Colossal Cleaned Corpus)

 Filtered Common Crawl (English, quality filtering)

 *Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus: 2021. <http://arxiv.org/abs/2104.08758>.*

## 3. RefinedWeb

 Heavily filtered Common Crawl for high-quality text

 *The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only: 2023. <http://arxiv.org/abs/2306.01116>.*

 *Deduplicating Training Data Makes Language Models Better: 2022. <http://arxiv.org/abs/2107.06499>.*

## WHY DEDUPLICATE?

1. Reduces training time and costs
2. Increasingly harmful to model quality as parameter count increases (*only for web-based data?*)

 *Scaling Laws and Interpretability of Learning from Repeated Data: 2022. <http://arxiv.org/abs/2205.10487>.*

3. Reduces memorization

 *Extracting Training Data from Large Language Models: 2021. <http://arxiv.org/abs/2012.07805>.*

## How?

### Exact

Using W Suffix Array or hashing (W MD5, W SHA-1)

### N-Gram based

MinHash

### Locality-Sensitive Hashing

LSH (text-specific hashing)

# REMOVE LOW-QUALITY & OFFENSIVE CONTENT

1. Language detection
2. Offensive content filtering
3. Length-based filtering
4. Domain-specific filtering

📖 *Challenges in Detoxifying Language Models: 2021.* <http://arxiv.org/abs/2109.07445>.

📖 *Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus: 2021.* <http://arxiv.org/abs/2104.08758>.

## NORMALIZING THE CONTENT

Uses the GPT-2 tokenizer (Byte-Pair Encoding, BPE)

## MINHASH

📖 Broder, A.Z. 1997. On the Resemblance and Containment of Documents. *Proceedings. Compression and Complexity of SEQUENCES 1997* (Cat. No.97TB100171).

Approximates the Jaccard Similarity between the sets of  $n$ -grams ( $n = 5$ )

$$J(d_i, d_j) = \frac{|d_i \cap d_j|}{|d_i \cup d_j|}$$

with an empirical threshold of 0.8

🔧 Computes the hash  $h_s$  of each  $n$ -gram  $s$  using W Tabulation Hashing

Random value for each position and token in the vocabulary

🔧 Use the minimum (over  $h_s$ ) of  $b$  (20) hash functions for each of the  $r$  (450) buckets

Equal value in one bucket = potential match

$$P(d_i, d_j \mid \text{Jaccard}(d_i, d_j) = s_{i,j}) = 1 - (1 - s_{i,j}^b)^r$$

## BANNED URLS

| Category    | Description                                        | Number of links |
|-------------|----------------------------------------------------|-----------------|
| adult       | adult websites: from eroticism to hard pornography | 4516478         |
| phishing    | phishing websites, malwares, etc.                  | 42445           |
| dating      | dating websites                                    | 3829            |
| gambling    | online casino                                      | 1365            |
| filehosting | websites hosting files, videos, pictures, music    | 909             |
| ddos        | websites related to ddos attacks                   | 421             |
| agressif    | hate, racism, etc                                  | 390             |
| chat        | online chat websites                               | 244             |
| mixed adult | websites with some adult content                   | 153             |
| arjel       | French regulated gambling websites                 | 69              |

AGGREGATED LIST OF ABOUT 4.6M URLS – WITH SOME FALSE POSITIVES (WEB SITES HOSTING USER CONTENT) THAT WERE MANUALLY CLASSIFIED

## USING RULES

**Strict subword matching:** [http://foo\*\*bann\*\*.\*\*edsub-wo.rd\*\*bar.com/any/bar](http://foo<b>bann</b>.<b>edsub-wo.rd</b>bar.com/any/bar), matching words such as xvideos, groupsex;

**Hard whole word matching:** [http://www.foo.\*\*bannedword\*\*-bar.com](http://www.foo.<b>bannedword</b>-bar.com), with words such as porn, xxx, orgy;

**Soft words matching:** [http://www.foo.\*\*soft1\*\*-bar-\*\*soft2\*\*.com](http://www.foo.<b>soft1</b>-bar-<b>soft2</b>.com), with "softer" words such as sex, webcam, escort.

## REMOVING HIGH QUALITY DATA

☞ arxiv, AskUbuntu, StackOverflow, NIH Abstracts, Github, Ubuntu IRC, HackerNews, FreeLaw, Reddit, Europarl, United States Patents, Wikipedia

## 🔗 THE PILE (825 GB)

🔗 22 diverse datasets: CommonCrawl, books, Wikipedia, arXiv, GitHub, etc.

🔗 Replaced by the Common Pile in early 2025 (8TB)

📄 *The Common Pile v0.1: An 8TB Dataset of Public Domain and Openly Licensed Text: 2025. <http://arxiv.org/abs/2506.05209>.*

📄 *The Pile: An 800GB Dataset of Diverse Text for Language Modeling: 2020. <http://arxiv.org/abs/2101.00027>.*

| Component                      | Raw Size          | Weight | Epochs | Effective Size     | Mean Document Size |
|--------------------------------|-------------------|--------|--------|--------------------|--------------------|
| Pile-CC                        | 227.12 GiB        | 18.11% | 1.0    | 227.12 GiB         | 4.33 KiB           |
| PubMed Central                 | 90.27 GiB         | 14.40% | 2.0    | 180.55 GiB         | 30.55 KiB          |
| Books3 <sup>†</sup>            | 100.96 GiB        | 12.07% | 1.5    | 151.44 GiB         | 538.36 KiB         |
| OpenWebText2                   | 62.77 GiB         | 10.01% | 2.0    | 125.54 GiB         | 3.85 KiB           |
| ArXiv                          | 56.21 GiB         | 8.96%  | 2.0    | 112.42 GiB         | 46.61 KiB          |
| Github                         | 95.16 GiB         | 7.59%  | 1.0    | 95.16 GiB          | 5.25 KiB           |
| FreeLaw                        | 51.15 GiB         | 6.12%  | 1.5    | 76.73 GiB          | 15.06 KiB          |
| Stack Exchange                 | 32.20 GiB         | 5.13%  | 2.0    | 64.39 GiB          | 2.16 KiB           |
| USPTO Backgrounds              | 22.90 GiB         | 3.65%  | 2.0    | 45.81 GiB          | 4.08 KiB           |
| PubMed Abstracts               | 19.26 GiB         | 3.07%  | 2.0    | 38.53 GiB          | 1.30 KiB           |
| Gutenberg (PG-19) <sup>†</sup> | 10.88 GiB         | 2.17%  | 2.5    | 27.19 GiB          | 398.73 KiB         |
| OpenSubtitles <sup>†</sup>     | 12.98 GiB         | 1.55%  | 1.5    | 19.47 GiB          | 30.48 KiB          |
| Wikipedia (en) <sup>†</sup>    | 6.38 GiB          | 1.53%  | 3.0    | 19.13 GiB          | 1.11 KiB           |
| DM Mathematics <sup>†</sup>    | 7.75 GiB          | 1.24%  | 2.0    | 15.49 GiB          | 8.00 KiB           |
| Ubuntu IRC                     | 5.52 GiB          | 0.88%  | 2.0    | 11.03 GiB          | 545.48 KiB         |
| BookCorpus2                    | 6.30 GiB          | 0.75%  | 1.5    | 9.45 GiB           | 369.87 KiB         |
| EuroParl <sup>†</sup>          | 4.59 GiB          | 0.73%  | 2.0    | 9.17 GiB           | 68.87 KiB          |
| HackerNews                     | 3.90 GiB          | 0.62%  | 2.0    | 7.80 GiB           | 4.92 KiB           |
| YoutubeSubtitles               | 3.73 GiB          | 0.60%  | 2.0    | 7.47 GiB           | 22.55 KiB          |
| PhilPapers                     | 2.38 GiB          | 0.38%  | 2.0    | 4.76 GiB           | 73.37 KiB          |
| NIH ExPorter                   | 1.89 GiB          | 0.30%  | 2.0    | 3.79 GiB           | 2.11 KiB           |
| Enron Emails <sup>†</sup>      | 0.88 GiB          | 0.14%  | 2.0    | 1.76 GiB           | 1.78 KiB           |
| <b>The Pile</b>                | <b>825.18 GiB</b> |        |        | <b>1254.20 GiB</b> | <b>5.91 KiB</b>    |

## Sentiment Analysis

IMDB, SST, ...

"The movie was absolutely incredible! The acting was superb and I was completely engaged from start to finish." → **Positive**

## Question Answering

SQuAD, Natural Questions, ...

**Context:** The Great Wall of China was built over many centuries by different Chinese dynasties. The Ming Dynasty, which ruled from 1368 to 1644, constructed most of the wall that exists today.

**Question:** Which dynasty built most of the Great Wall of China that exists today?  
→ **Ming Dynasty**

## Text Classification

AG News, DBpedia, ...

"Apple Inc. announced today that it has achieved record-breaking quarterly revenue of \$120 billion, driven primarily by strong iPhone sales in emerging markets." → **Business**

## 🔧 Task-specific dataset compilations for instruction tuning

### 1. FLAN (Finetuned Language Net)

- 🔧 1,800+ tasks from 400+ datasets

- 🔧 Formatted as instructions (e.g., "Translate to French: ...", "Summarize: ...")

- 📖 *Wei, J. et al. 2021. Finetuned Language Models Are Zero-Shot Learners.*

### 2. SuperNaturalInstructions

- 🔧 1,600+ NLP tasks with expert-written instructions

- 📖 *Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks: 2022. <http://arxiv.org/abs/2204.07705>.*

### 3. 🔗 P3 (Public Pool of Prompts)

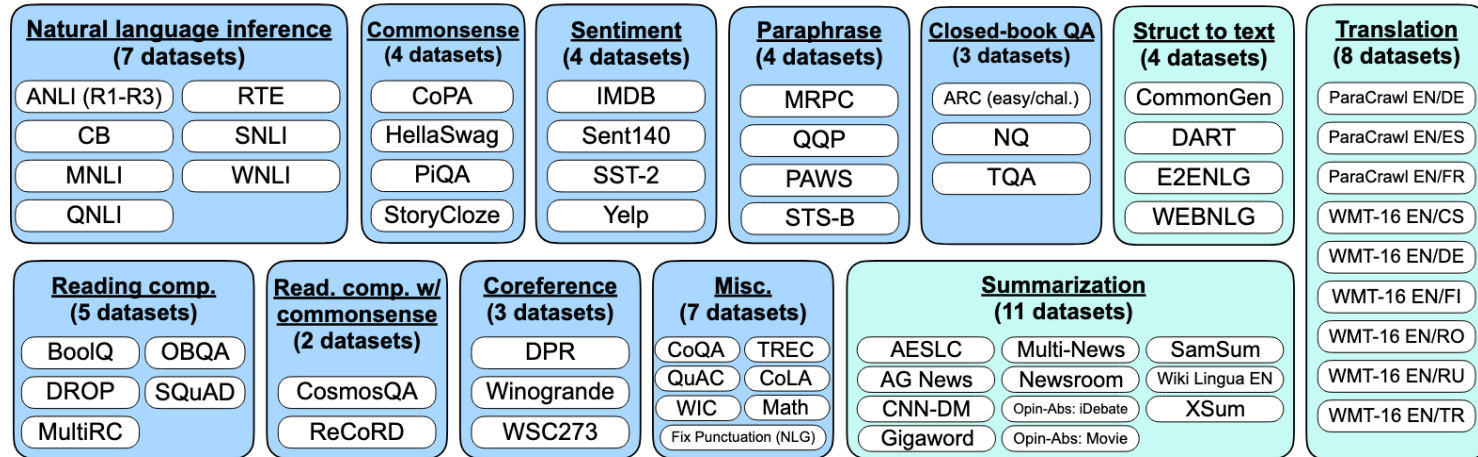
- 🔧 170+ datasets with multiple prompt templates

- 🔧 Generalizes the idea of prompts for few-shot learning

- 📖 *Sanh, V. et al. 2022. Multitask Prompted Training Enables Zero-Shot Task Generalization.*

# TRAINING ► EXAMPLE: FLAN

Wei, J. et al. 2021. *Finetuned Language Models Are Zero-Shot Learners*.



## Premise

Russian cosmonaut Valery Polyakov set the record for the longest continuous amount of time spent in space, a staggering 438 days, between 1994 and 1995.

## Hypothesis

Russians hold the record for the longest stay in space.

## Target

Entailment  
Not entailment



Options:  
- yes  
- no

## Template 1

<premise>  
Based on the paragraph above, can we conclude that  
<hypothesis>?  
<options>

## Template 2

<premise>  
Can we infer the following?  
<hypothesis>  
<options>

## Template 3

Read the following and determine if the hypothesis can be inferred from the premise:  
Premise: <premise>  
Hypothesis: <hypothesis>  
<options>

## Template 4, ...

## BERT and GPT-1

 **BookCorpus** (self-published books from Smashwords, 800M words)

(BERT only) English Wikipedia (2,500M words)

 Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

 Radford, A. et al. 2018. *Improving Language Understanding by Generative Pre-Training*.

## neoBERT

 NeoBERT: A Next-Generation BERT: 2025. <http://arxiv.org/abs/2502.19587>.

Uses  RefinedWeb

 The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only: 2023. <http://arxiv.org/abs/2306.01116>.

## GPT-3

 Brown, T.B. et al. 2020. *Language Models Are Few-Shot Learners*.

1. Filtered  **Common Crawl** (de-duplication)
2. WebText2 (filtered from outbound links on Reddit)
3. Books1 and Books2 (two internet-based books corpora)
4. English Wikipedia



## GPT-3 + FLAN (Instruct models)

 Wei, J. et al. 2021. *Finetuned Language Models Are Zero-Shot Learners*.



GPT-3 finetuned on FLAN tasks

## 🧪 GPT-3 + FLAN (Instruct models)

📖 Wei, J. et al. 2021. *Finetuned Language Models Are Zero-Shot Learners*.

🔧 GPT-3 finetuned on FLAN tasks

🔧 Improves performance over unseen tasks

### Finetune on many tasks (“instruction-tuning”)

#### Input (Commonsense Reasoning)

Here is a goal: Get a cool sleep on summer days.

How would you accomplish this goal?

OPTIONS:

-Keep stack of pillow cases in fridge.

-Keep stack of pillow cases in oven.

#### Target

keep stack of pillow cases in fridge

#### Input (Translation)

Translate this sentence to Spanish:

The new office building was built in less than three months.

#### Target

El nuevo edificio de oficinas se construyó en tres meses.

Sentiment analysis tasks

Coreference resolution tasks

...

### Inference on unseen task type

#### Input (Natural Language Inference)

Premise: At my age you will probably have learnt one lesson.

Hypothesis: It's not certain how many lessons you'll learn by your thirties.

Does the premise entail the hypothesis?

OPTIONS:

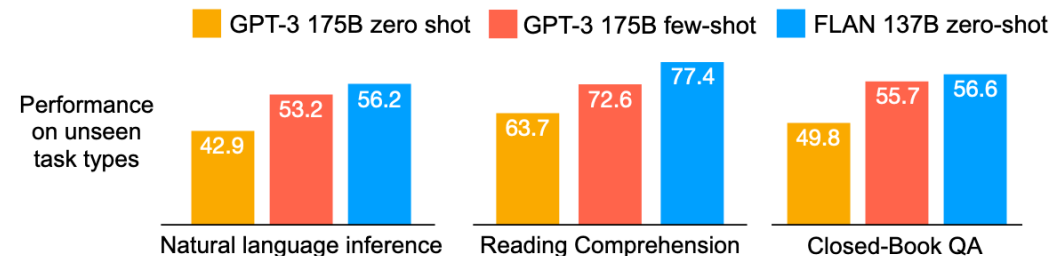
-yes

-it is not possible to tell

-no

#### FLAN Response

It is not possible to tell

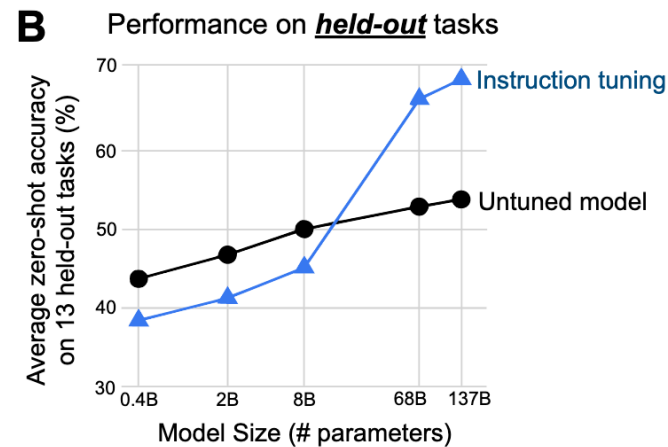
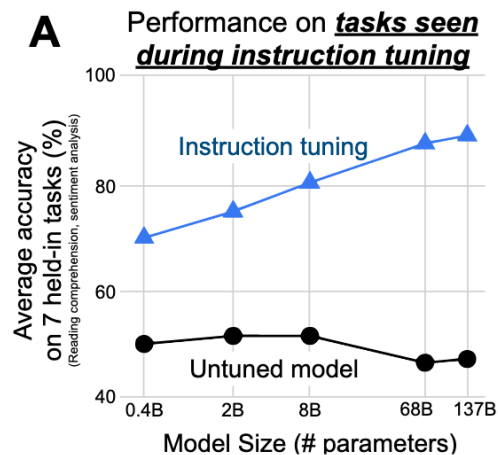


## 🧪 GPT-3 + FLAN (Instruct models)

📖 Wei, J. et al. 2021. *Finetuned Language Models Are Zero-Shot Learners*.

🧠 GPT-3 finetuned on FLAN tasks

🧠 This is especially important when increasing the size of models



## DOLMINO (USED TO TRAIN OLMo)

 2 OLMo 2 Furious: 2025. <http://arxiv.org/abs/2501.00656>.

## DOLMINO (USED TO TRAIN OLMo)

📖 2 OLMo 2 Furious: 2025. <http://arxiv.org/abs/2501.00656>.

### PRETRAINING DATA: OLMo 2 Mix 1124

| Source                                          | Type             | Tokens       | Words        | Bytes         | Docs         |
|-------------------------------------------------|------------------|--------------|--------------|---------------|--------------|
| <b>Pretraining ♦ OLMo 2 1124 Mix</b>            |                  |              |              |               |              |
| DCLM-Baseline                                   | Web pages        | 3.71T        | 3.32T        | 21.32T        | 2.95B        |
| StarCoder<br>filtered version<br>from OLMoE Mix | Code             | 83.0B        | 70.0B        | 459B          | 78.7M        |
| peS2o<br>from Dolma 1.7                         | Academic papers  | 58.6B        | 51.1B        | 413B          | 38.8M        |
| arXiv                                           | STEM papers      | 20.8B        | 19.3B        | 77.2B         | 3.95M        |
| OpenWebMath                                     | Math web pages   | 12.2B        | 11.1B        | 47.2B         | 2.89M        |
| Algebraic Stack                                 | Math proofs code | 11.8B        | 10.8B        | 44.0B         | 2.83M        |
| Wikipedia & Wikibooks<br>from Dolma 1.7         | Encyclopedic     | 3.7B         | 3.16B        | 16.2B         | 6.17M        |
| <b>Total</b>                                    |                  | <b>3.90T</b> | <b>3.48T</b> | <b>22.38T</b> | <b>3.08B</b> |

## DOLMINO (USED TO TRAIN OLMo)

📖 2 OLMo 2 Furious: 2025. <http://arxiv.org/abs/2501.00656>.

## MID-TRAINING DATA: DOLMINO MIX 1124

| Source                                                    | Type             | Tokens        | Words         | Bytes        | Docs          |
|-----------------------------------------------------------|------------------|---------------|---------------|--------------|---------------|
| Mid-Training ♦ Dolmino High Quality Subset                |                  |               |               |              |               |
| DCLM-Baseline<br>FastText top 7%<br>FineWeb ≥ 2           | High quality web | 752B          | 670B          | 4.56T        | 606M          |
| FLAN<br>from Dolma 1.7<br>decontaminated                  | Instruction data | 17.0B         | 14.4B         | 98.2B        | 57.3M         |
| peS2o<br>from Dolma 1.7                                   | Academic papers  | 58.6B         | 51.1B         | 413B         | 38.8M         |
| Wikipedia & Wikibooks<br>from Dolma 1.7                   | Encyclopedic     | 3.7B          | 3.16B         | 16.2B        | 6.17M         |
| Stack Exchange<br>09/30/2024 dump<br>curated Q&A data     | Q&A              | 1.26B         | 1.14B         | 7.72B        | 2.48M         |
| <b>High quality total</b>                                 |                  | <b>832.6B</b> | <b>739.8B</b> | <b>5.09T</b> | <b>710.8M</b> |
| Mid-training ♦ Dolmino Math Mix                           |                  |               |               |              |               |
| TuluMath                                                  | Synthetic math   | 230M          | 222M          | 1.03B        | 220K          |
| Dolmino SynthMath                                         | Synthetic math   | 28.7M         | 35.1M         | 163M         | 725K          |
| TinyGSM-MIND                                              | Synthetic math   | 6.48B         | 5.68B         | 25.52B       | 17M           |
| MathCoder2<br>Synth Books<br>Ajibawa-2023<br>M-A-P Matrix | Synthetic Math   | 3.87B         | 3.71B         | 18.4B        | 2.83M         |
| Metamath<br>OWM-filtered                                  | Math             | 84.2M         | 76.6M         | 741M         | 383K          |
| CodeSearchNet<br>OWM-filtered                             | Code             | 1.78M         | 1.41M         | 29.8M        | 7.27K         |
| GSM8K<br>Train split                                      | Math             | 2.74M         | 3.00M         | 25.3M        | 17.6K         |
| <b>Math total</b>                                         |                  | <b>10.7B</b>  | <b>9.73B</b>  | <b>45.9B</b> | <b>21.37M</b> |

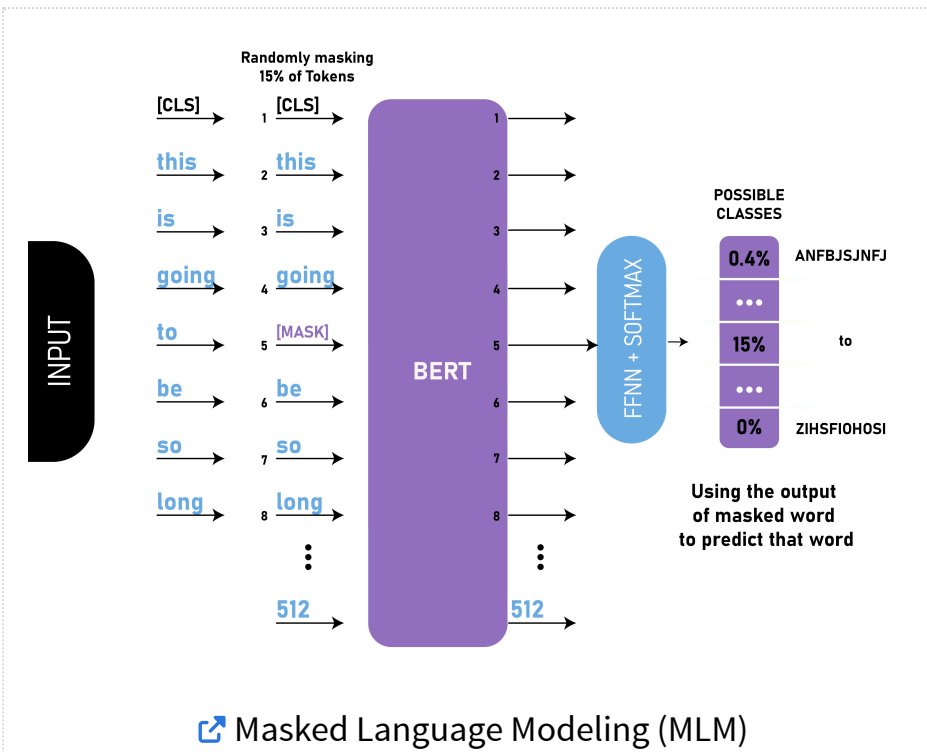
### KEY TAKEAWAYS

1. Diverse and high-quality data is crucial for training effective LLMs
2. Data cleaning (deduplication, filtering) improves model performance
3. Combining multiple data sources can enhance model generalization
4. Instruction-tuning *and* scale is key to generalization

# TRAINING

## TRAINING ENCODERS

# TRAINING ► MASKED-LANGUAGE MODELING



Original text

$\mathbf{t}_{1 \dots L}$

Corrupted text

$$\tilde{\mathbf{t}}_i = \begin{cases} \kappa(\mathbf{t}_i) & \text{with probability } p_{\text{mask}} \\ \mathbf{t}_i & \text{otherwise} \end{cases}$$

Loss

$$\mathcal{L} = - \sum_{i \in M} \log p_{\theta}(\mathbf{t}_i | \tilde{\mathbf{t}}_{1 \dots L})$$

where  $M$  is the set of masked token positions

### ⚠ Problem statement

We have a corrupted text  $\tilde{\mathbf{t}}_{1..L}$

How to compute  $p_{\theta}(\mathbf{t}_i | \tilde{\mathbf{t}}_{1..L})$ ?

1. We compute the contextualized representation  $\tilde{\mathbf{x}}_{1..L}^{(N)}$   
Using a Transformer decoder (masked self-attention, with auto-regressive masking)
2. Then use a linear layer + softmax (**un-embedding matrix**) to get probabilities over the vocabulary

$$p_{\theta}(\mathbf{t}_i | \tilde{\mathbf{t}}_{1..L}) = \text{softmax}(\mathbf{W}_u \tilde{\mathbf{x}}_i^{(N)} + \mathbf{b}_u)$$

## BERT-style masking

 Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

1. 15% of tokens selected for possible masking
2. Of these:
  - 80% replaced with [MASK]
  - 10% replaced with a *random token*
  - 10% left unchanged

## BERT-style masking

 Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

1. 15% of tokens selected for possible masking
2. Of these:
  - 80% replaced with [MASK]
  - 10% replaced with a *random token*
  - 10% left unchanged

## OTHER MASKING STRATEGIES

### 1. Whole Word Masking

 If any subword of a word is masked, mask the entire word

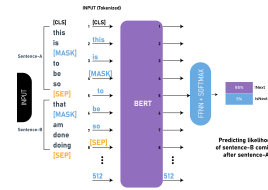
 Cui, Y. et al. 2021. *Pre-Training with Whole Word Masking for Chinese BERT*.

Models: BERT, RoBERTa

### 2. Frequency-Based Masking

 More frequent words have a lower probability of being masked

🔑 Given two sentences A and B, predict if B follows A in the original text



🔗 Masked Language Modeling (MLM)

## Input

[CLS]  $\mathbf{t}_A$  [SEP]  $\mathbf{t}_B$  [SEP]

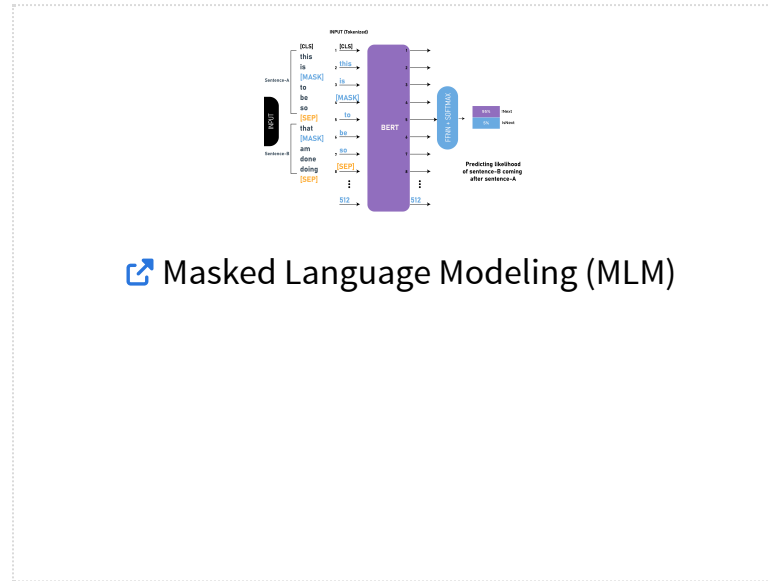
## Loss

$$\mathcal{L} = -\log p_{\theta}(y|\mathbf{t}_A, \mathbf{t}_B)$$

where  $y$  is 1 if B follows A, 0 otherwise

# TRAINING ► TEXT/SENTENCE ENCODING

☞ Given two sentences A and B, predict if B follows A in the original text



☞ Masked Language Modeling (MLM)

## Input

[CLS]  $\mathbf{t}_A$  [SEP]  $\mathbf{t}_B$  [SEP]

## Loss

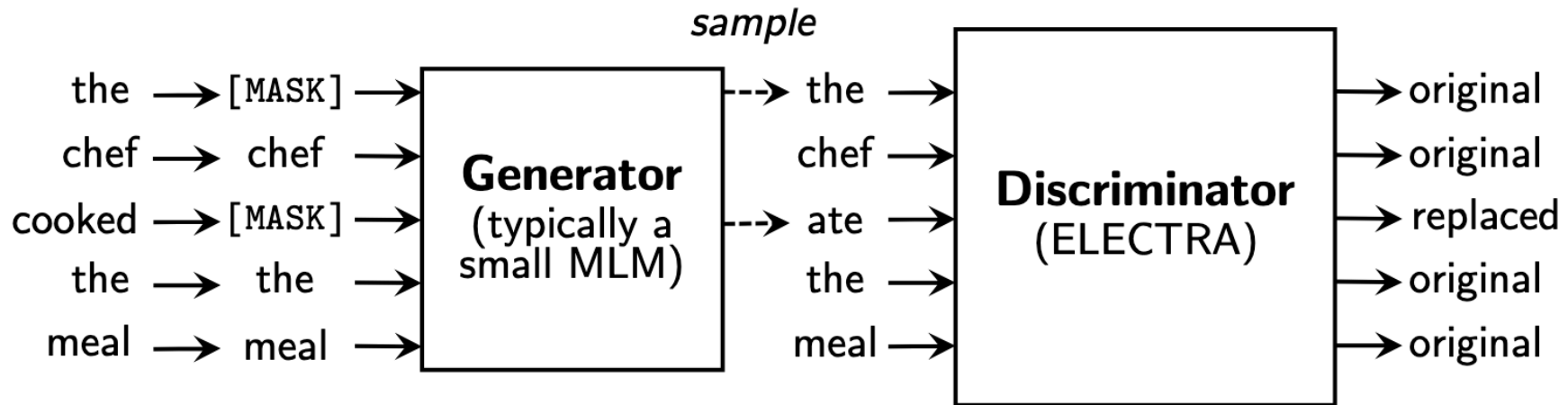
$$\mathcal{L} = -\log p_{\theta}(y|\mathbf{t}_A, \mathbf{t}_B)$$

where  $y$  is 1 if B follows A, 0 otherwise

☞ Not really used in practice (further works have shown that it did not change anything)

## 💡 Motivation

- 🔧 Instead of masking tokens, replace them with plausible alternatives
- 🔧 Model learns to detect replaced tokens, encouraging deeper understanding



📖 Clark, K. et al. 2019. *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators*.

## The generator

Masked language model (like BERT)

## The discriminator

Predicts whether each token was replaced by the generator

## Used in Electra and DeBERTaV3

 Clark, K. et al. 2019. *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators*.

 He, P. et al. 2022. *DeBERTaV3: Improving DeBERTa Using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing*.

 The generator/discriminator share the same embeddings

 The generator is smaller (e.g. half the number of layers in DeBERTaV3) than the discriminator

 More efficient than MLM (each token contributes to the loss)

 Better performance with less compute

 Requires training two models (generator + discriminator)

## TWO MAIN TRAINING PROCEDURES

- Generative: Masked Language Modelling: with special tricks!
- Discriminative: ELECTRA-style

🔑 No clear winner

📖 Sanh, V. et al. 2019. *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*.

Compares DeBERTaV3 and ModernBERT

🔑 ModernBERT preferred **because of its long context, faster training, inference speed**

## TWO MAIN TRAINING PROCEDURES

- Generative: Masked Language Modelling: with special tricks!
- Discriminative: ELECTRA-style

🔑 No clear winner

📖 Sanh, V. et al. 2019. *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*.

Compares DeBERTaV3 and ModernBERT

🔑 ModernBERT preferred **because of its long context, faster training, inference speed**

**simpler (MLM) methods are preferred**

# TRAINING

## TRAINING DECODERS

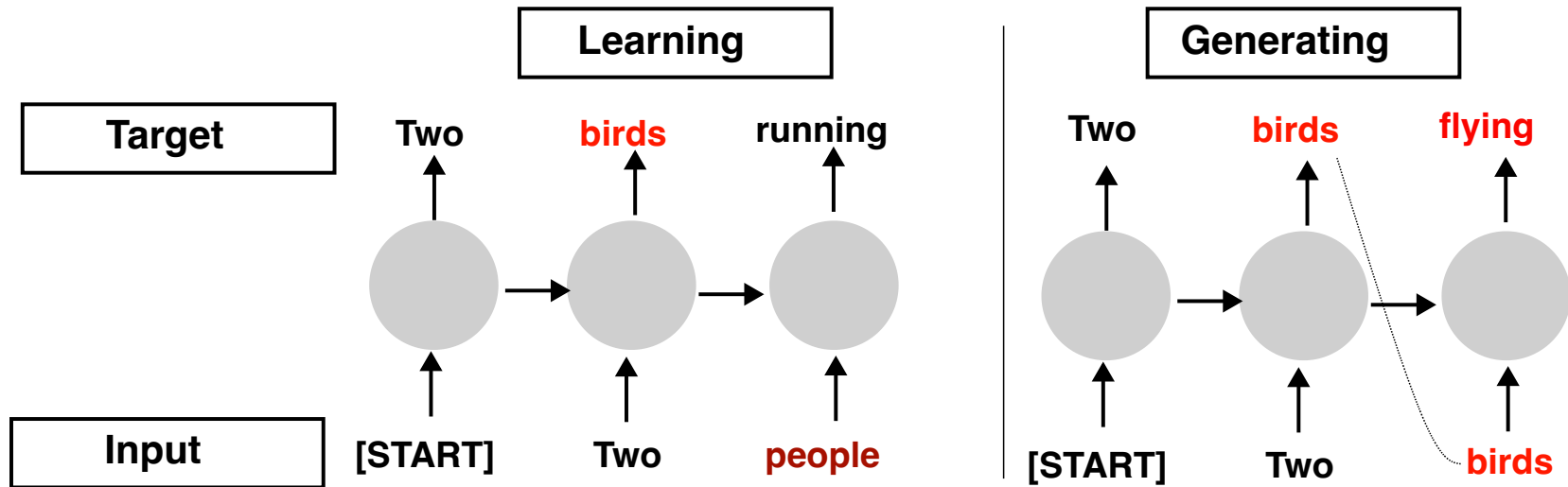
# CLM = MAXIMUM LIKELIHOOD ESTIMATION

Given a sequence of tokens  $\mathbf{t}_{1..L}$ , maximize the likelihood:

$$\mathcal{L} = - \sum_{i=1}^L \log p_{\theta}(\mathbf{t}_i | \mathbf{t}_{1..i-1})$$

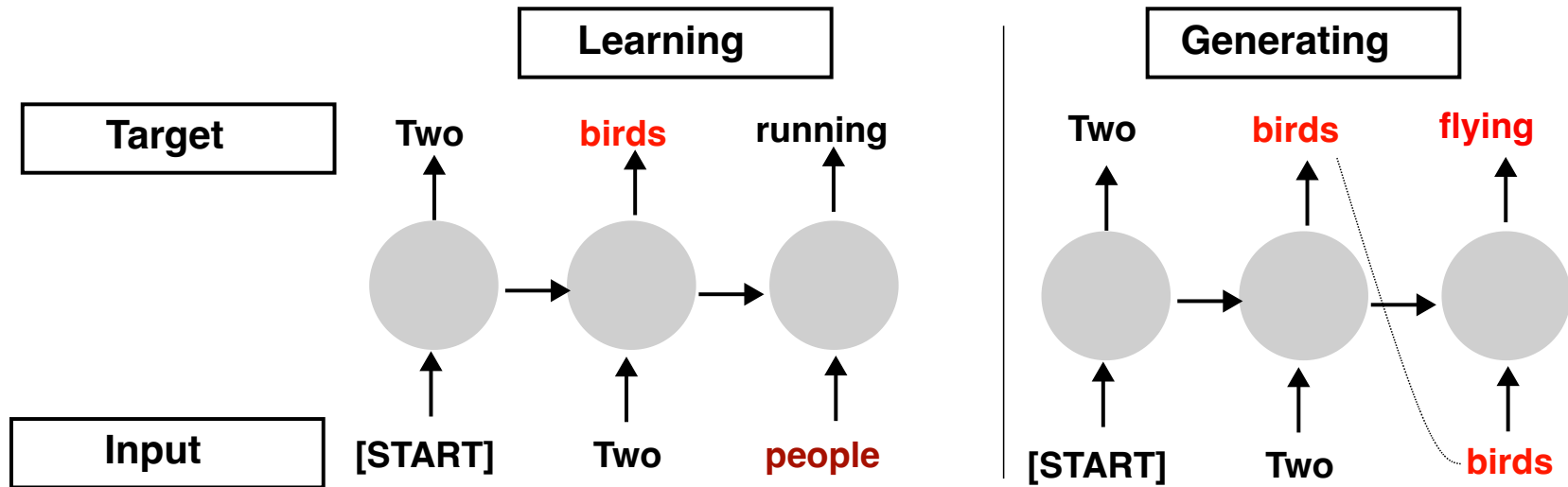
# TEACHER FORCING

Ground truth  
Two people running



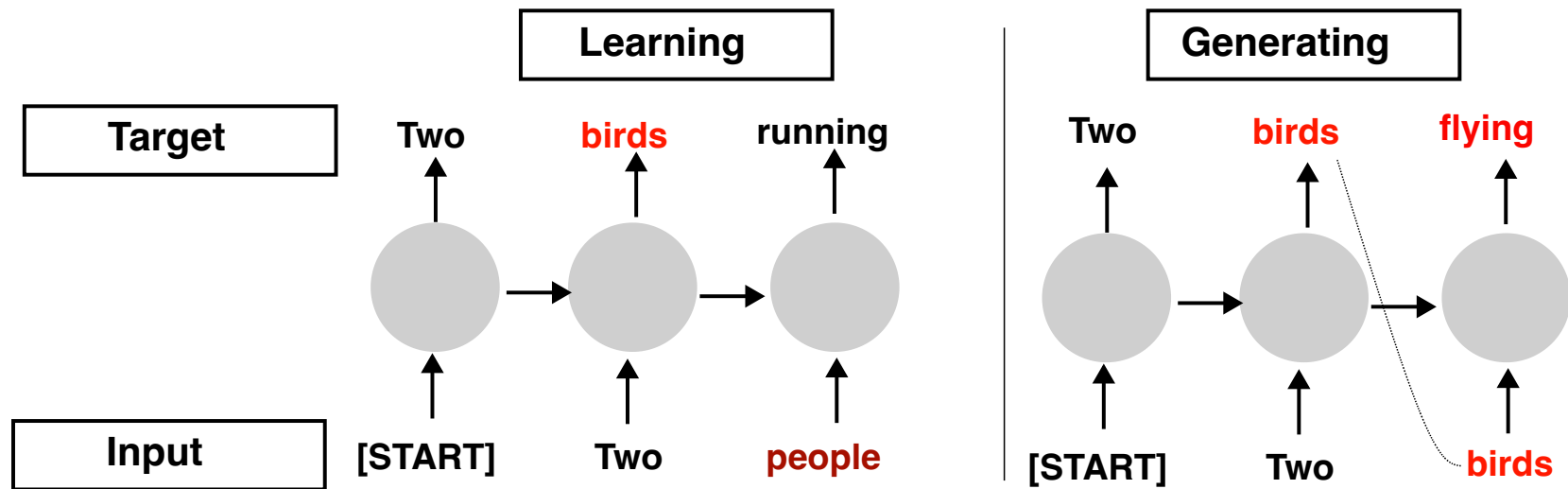
# TEACHER FORCING

Ground truth  
Two people running



# TEACHER FORCING

Ground truth  
Two **people** running

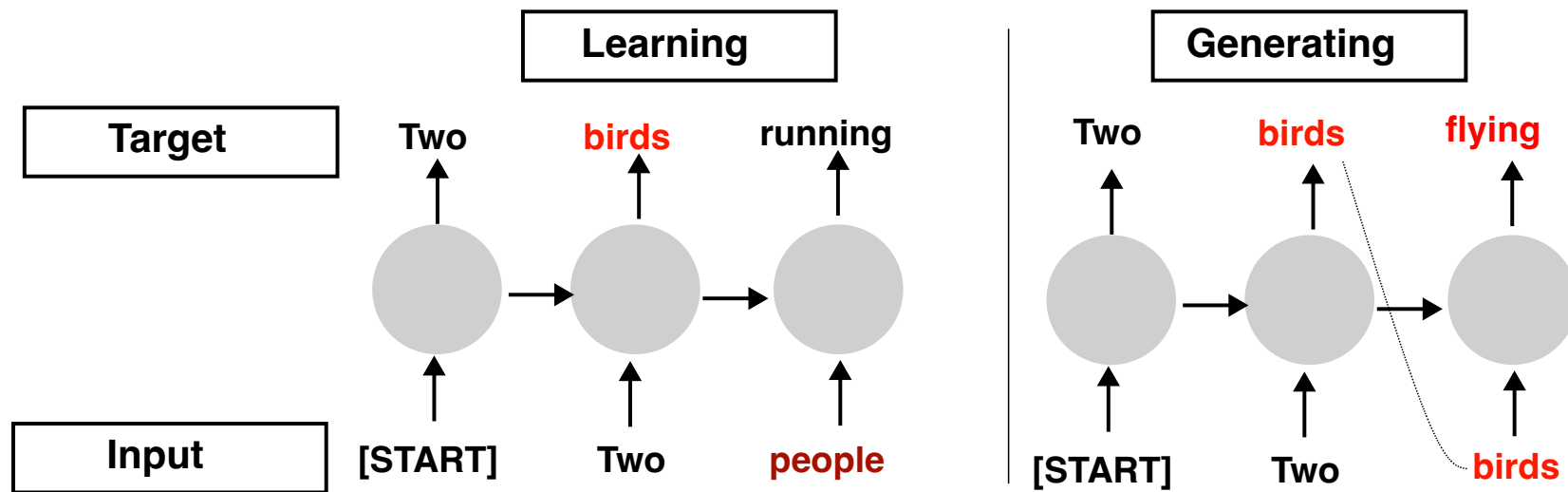


So what's wrong ?

## TEACHER FORCING

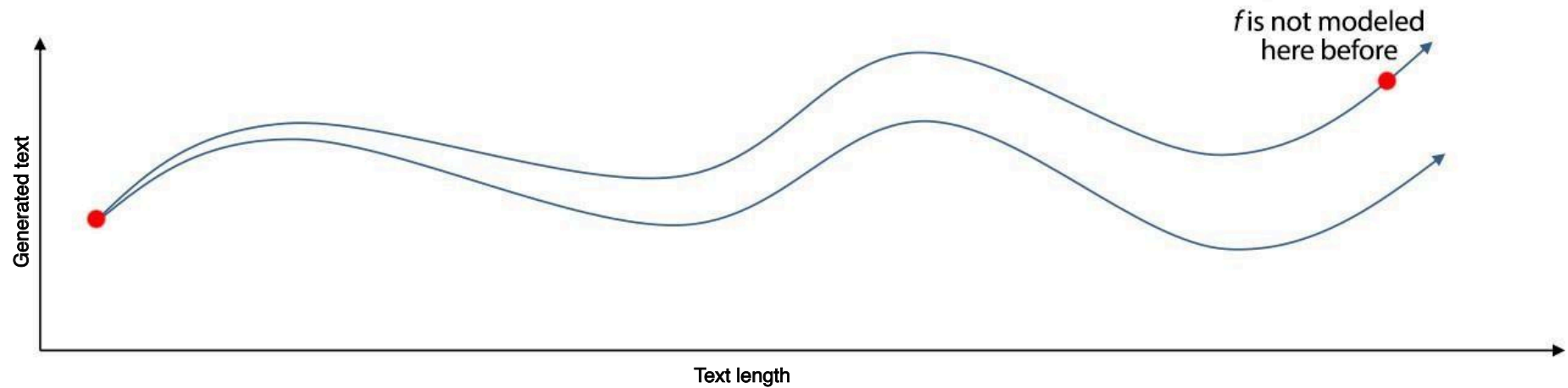
Ground truth

Two people running



So what's wrong ?

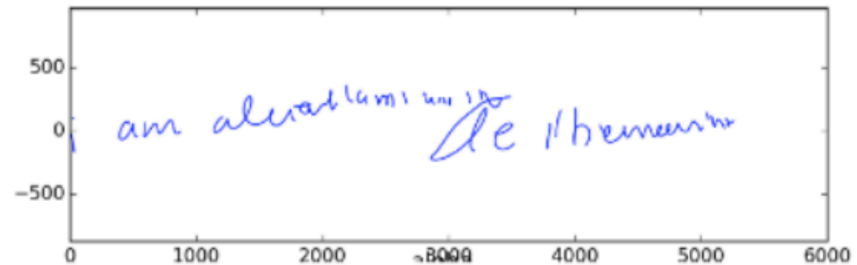
The model is never exposed to a series of mistakes when learning - at inference time... this might lead to problems



The space of generate sentences is very large

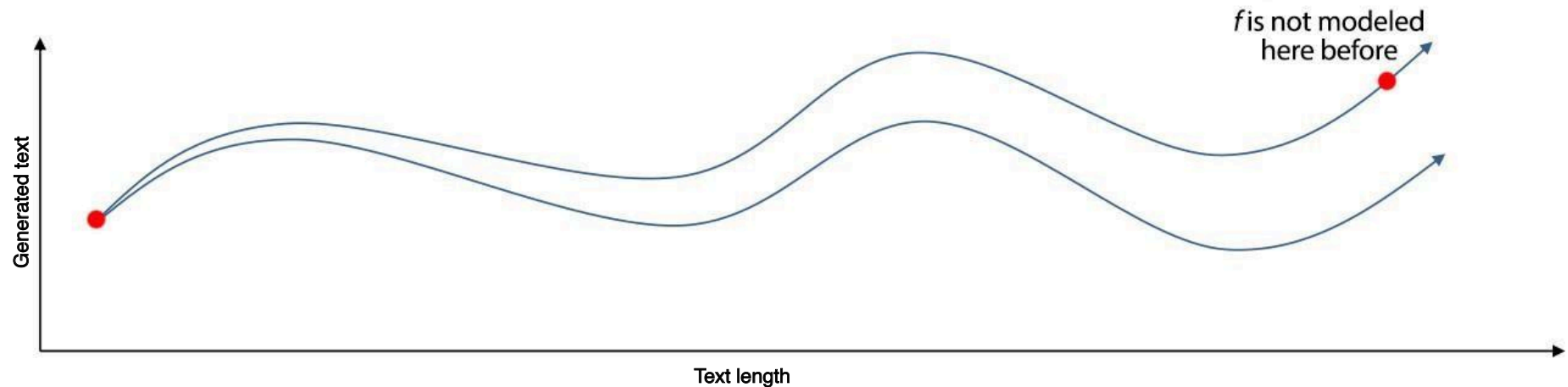
☞ Easy to be at a stage where the model don't know!

⚠ This leads to degenerated texts



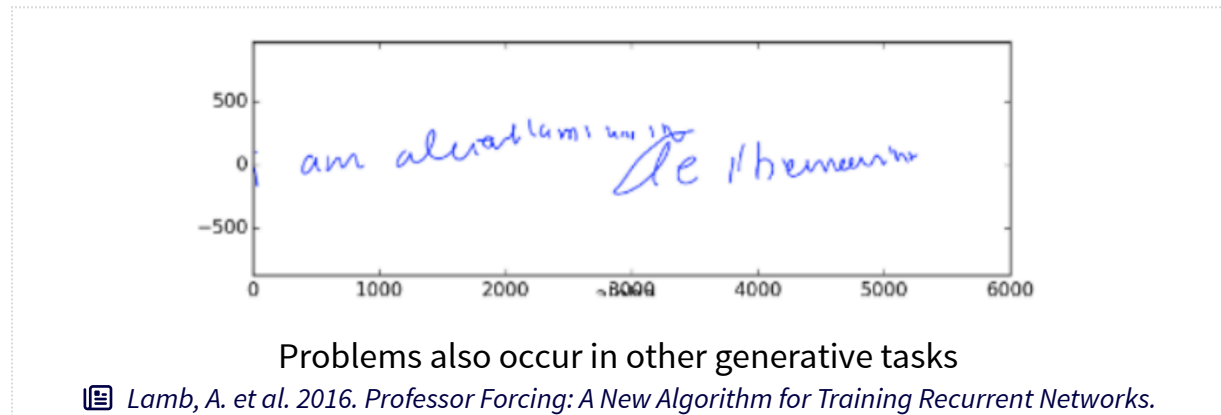
Problems also occur in other generative tasks

📖 Lamb, A. et al. 2016. Professor Forcing: A New Algorithm for Training Recurrent Networks.



The space of generate sentences is very large  
 ☞ Easy to be at a stage where the model don't know!

⚠ This leads to degenerated texts



**In practice, not a problem for LLMs trained on massive data... but still relevant for alignment phases (or when training with small datasets)!**

## ⚠ Problem statement

We have a prefix  $\mathbf{t}_{1..L-1}$

How to compute  $p_{\theta}(\mathbf{t}_i | \mathbf{t}_{1..i-1})$ ?

1. We compute the contextualized representation  $\mathbf{x}_{1..L-1}^{(N)}$   
Using a Transformer decoder (masked self-attention, with auto-regressive masking)
2. Then use a linear layer + softmax (**un-embedding matrix**) to get probabilities over the vocabulary

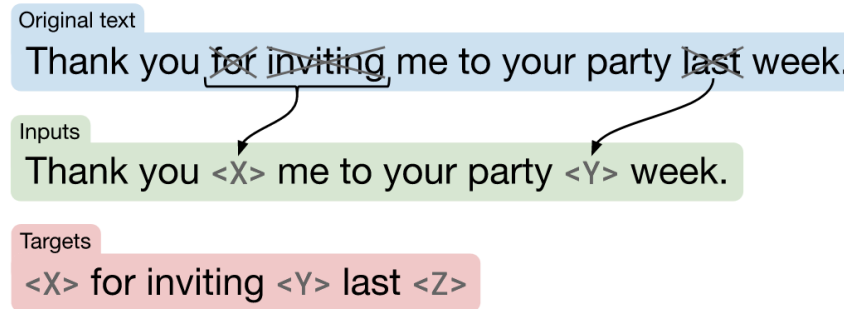
$$p_{\theta}(\mathbf{t}_i | \mathbf{t}_{1..i-1}) = \text{softmax}(\mathbf{W}_u \mathbf{x}_{i-1}^{(N)} + \mathbf{b}_u)$$

# TRAINING ► ALTERNATIVE LOSSES: SCRAMBLED TOKEN PREDICTION

Instead of predicting the next token, predict a randomly chosen token from the sequence  
Encourages the model to understand the overall context rather than just local continuity

## Used in T5

 Raffel, C. et al. 2019. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*.

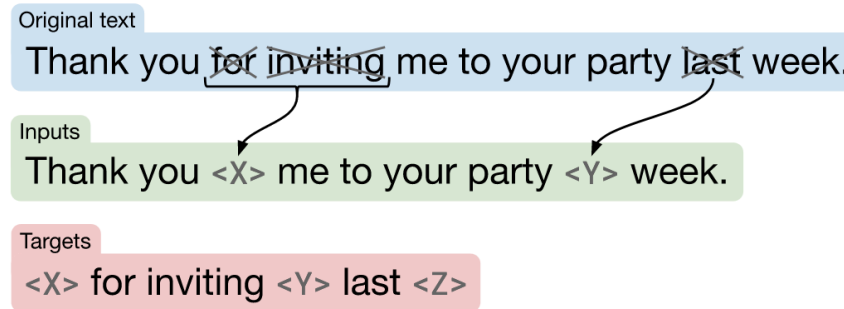


# TRAINING ► ALTERNATIVE LOSSES: SCRAMBLED TOKEN PREDICTION

Instead of predicting the next token, predict a randomly chosen token from the sequence  
Encourages the model to understand the overall context rather than just local continuity

## 🧪 Used in T5

📖 Raffel, C. et al. 2019. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*.



🔔 Not really used in practice in LLMs (CLM is simpler and works well enough)

## WHY RL FOR LLMs?

🔧 Instruction tuning (FLAN) allows us to enable language models to follow novel user instructions that are not seen during finetuning (this is what the users of LMs want!)

### 👉 Limitations of MLE for instruction following

- Getting data is expensive, especially for very complex tasks
- Some tasks may not have one single acceptable output (Exposure bias)
- Does not directly model human preferences

## WHY RL FOR LLMs?

☞ Instruction tuning (FLAN) allows us to enable language models to follow novel user instructions that are not seen during finetuning (this is what the users of LMs want!)

### 👉 Limitations of MLE for instruction following

- Getting data is expensive, especially for very complex tasks
- Some tasks may not have one single acceptable output (Exposure bias)
- Does not directly model human preferences

## RLHF: REINFORCEMENT LEARNING FROM HUMAN FEEDBACK

- ☞ Train a reward model from human preferences
- ☞ Use RL to optimize the language model policy

## STEP 1: SUPERVISED FINE-TUNING

1. Collect high-quality demonstrations
2. Fine-tune pre-trained LM with supervised learning
3. Creates initial *policy*  $\pi_{\theta_{SFT}}$

## STEP 2: REWARD MODEL TRAINING

1. Sample prompts, generate multiple responses
2. Collect human preference comparisons
3. Train reward model  $r_{\phi}(\mathbf{t})$  to predict preferences

## STEP 3: RL OPTIMIZATION

1. Use RL algorithm (PPO) to optimize policy
2. Maximize expected reward (while staying close to initial policy)  
 PPO, GRPO, DPO...
3. (go back to step 2)

📖 Schulman, J. et al. 2017. Proximal Policy Optimization Algorithms.

## KEY IDEA

🔧 Used in ChatGPT, InstructGPT

📖 Training Language Models to Follow Instructions with Human Feedback: 2022. <http://arxiv.org/abs/2203.02155>.

Optimize a PPO-like loss

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[ (r_{\theta}(x, y) - V_{\phi}(x, y)) - \beta \log \left( \pi_{\phi}^{\text{RL}}(y | x) / \pi^{\text{SFT}}(y | x) \right) \right] + \\ \gamma E_{x \sim D_{\text{pretrain}}} \left[ \log \left( \pi_{\phi}^{\text{RL}}(x) \right) \right]$$

🔧  $\pi_{\phi}^{\text{RL}}$  the trained policy

🔧  $r_{\theta}(x, y)$  = reward model score,  $V_{\phi}(x, y)$  the value function

🔧  $\text{KL}(\pi_{\theta} || \pi^{\text{ref}})$ : KL divergence from reference policy (prevents mode collapse)

📖 *Direct Preference Optimization: Your Language Model Is Secretly a Reward Model: 2023.* <http://arxiv.org/abs/2305.18290>.

## KEY INSIGHT

🔑 Skip the reward model! Optimize policy directly from preferences

## FROM RL TO SUPERVISED LEARNING

🔑 DPO shows PPO optimization is equivalent to:

$$\mathcal{L}_{DPO}(\pi_{\theta}) = -\mathbb{E}_{(x, y_+, y_-)} \left[ \log \sigma \left( \beta \log \frac{\pi_{\theta}(y_+ | x)}{\pi^{ref}(y_+ | x)} - \beta \log \frac{\pi_{\theta}(y_- | x)}{\pi^{ref}(y_- | x)} \right) \right]$$

🔑 where  $y_+$  is preferred over  $y_-$  (labelled data) and  $\sigma$  is the sigmoid function

- ➕ Simpler: No reward model, no RL training loop
- ➕ More stable: Standard supervised learning
- ➕ Memory efficient: Only need policy and reference models

## CAN BE EVEN SIMPLIFIED FURTHER

📖 *Llama 2: Open Foundation and Fine-Tuned Chat Models: 2023.* <http://arxiv.org/abs/2307.09288>.

$$\mathcal{L}_{\text{ranking}} = -\log (\sigma (r_{\theta} (x, y_c) - r_{\theta} (x, y_r) - m(r)))$$

## PPO

- ➕ More flexible
  - ☞ Can incorporate non-preference rewards
  - ☞ Can do online learning
- ➖ Complex training
  - ☞ 4 models needed
  - ☞ Unstable, requires tuning
- ➖ Computationally expensive

## USED IN

- ☞ InstructGPT, ChatGPT

## DPO

- ➕ Simple training
  - ☞ Direct supervised learning
  - ☞ Only 2 models needed
- ➕ More stable
  - ☞ Less hyperparameter tuning
- ➖ Limited to preference data
  - ☞ Cannot use arbitrary reward functions

## USED IN

- ☞ Mistral models

## MOTIVATION

- ☞ PPO is complex and unstable
- ☞ DPO limited to offline preference data
- ☞ Can we get PPO's flexibility with DPO's simplicity?

## KEY IDEAS

- 📖 *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models: 2024.* <https://arxiv.org/abs/2402.03300v3>.
- 📖 *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model: 2024.* <http://arxiv.org/abs/2405.04434>.

1. Generate multiple responses per prompt (a group)
2. No separate value model needed: Use group statistics for advantage estimation

$$\text{☞ } \hat{A}_i = \frac{r_i - \text{mean}(r_{\text{group}})}{\text{std}(r_{\text{group}})}$$

- ☑ Simpler than PPO (no value model)
- ☑ Can use arbitrary reward functions (unlike DPO)
- ☑ More stable than PPO

**TRAINING**  
**FINETUNING**

 Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

## WHY FINETUNING?

1. Pre-trained models learn general language understanding
2. Finetuning adapts them to specific downstream tasks
3. Much faster and cheaper than training from scratch
4. Requires much less task-specific data

📖 Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

## WHY FINETUNING?

1. Pre-trained models learn general language understanding
2. Finetuning adapts them to specific downstream tasks
3. Much faster and cheaper than training from scratch
4. Requires much less task-specific data

## THE PROCESS

1. Start with pre-trained model (BERT, GPT, T5, etc.)
2. Add task-specific head (if needed, e.g. for encoders)
3. Train on task-specific data with lower learning rate
4. Optionally freeze some layers to prevent catastrophic forgetting

## ENCODER MODELS (BERT-LIKE)

- Text Classification
  - 🔑 Sentiment analysis, topic classification
- Token Classification
  - 🔑 Named Entity Recognition (NER), POS tagging
- Question Answering
  - 🔑 Extractive QA (SQuAD)
- Sentence Similarity
  - 🔑 Semantic textual similarity

## DECODER MODELS (GPT-LIKE)

- Text Generation
  - 🔑 Completion, creative writing
- Instruction Following
  - 🔑 Chat, task completion
- Code Generation
  - 🔑 Code completion, debugging

## ENCODER-DECODER (T5-LIKE)

- Translation, Summarization
- Text-to-text tasks

## LEARNING RATE

- Use lower learning rates than pre-training
- Typical:  $1e-5$  to  $5e-5$  for full finetuning
- Use learning rate warmup

## REGULARIZATION

- Weight decay (0.01 typical)
- Gradient clipping
- Dropout (if not already in model)

## DATA CONSIDERATIONS

- More data = better results (but diminishing returns)
- Balance dataset classes
- Use data augmentation when applicable
- Monitor for overfitting on small datasets

# TEXT GENERATION

## Understanding Contextual Generation is Key

- LLMs are multitask learners & problem solvers via prompting
- Evaluation of linguistic abilities of LLMs requires generation  
Agreement, proper word order, semantic consistency etc.
- Good artificial data helps ML  
Privacy, confidentiality, distillation / data augmentation, artificial text detection
- "Think" relies on good generation  
A key aspect of recent LLMs architectures

# TEXT GENERATION

## BASICS

# TEXT GENERATION ► A LANGUAGE MODEL IS A DISTRIBUTION

Language is a probability distribution over tokens

$$p_{\theta}(\mathbf{t}_{1:L})$$

☞ The sample space of sequences of tokens = Kleene closure of  $\mathcal{T}$

$$\mathbf{t} \in \bigcup_{L=1}^{\infty} \mathcal{T}^L$$

☞ Well, not so true (because of the EOS token), but close enough...

## LOCAL NORMALIZATION

☞ Factorized as product of conditionals

$$p_{\theta}(\mathbf{t}_{1:L}) = \prod_{i=1}^L p_{\theta}(\mathbf{t}_i | \mathbf{t}_{1:i-1})$$

☞ Notice that there is a twist: the length  $L$  is also a random variable!  
= Not the real Kleen closure of  $\mathcal{T}$

## TWO MAIN TYPES OF "GENERATION"

### Decoding

Find the most probable text  $\mathbf{t}_{1:L}$

☞ Useful for translation, summarization, question answering...

### Sampling

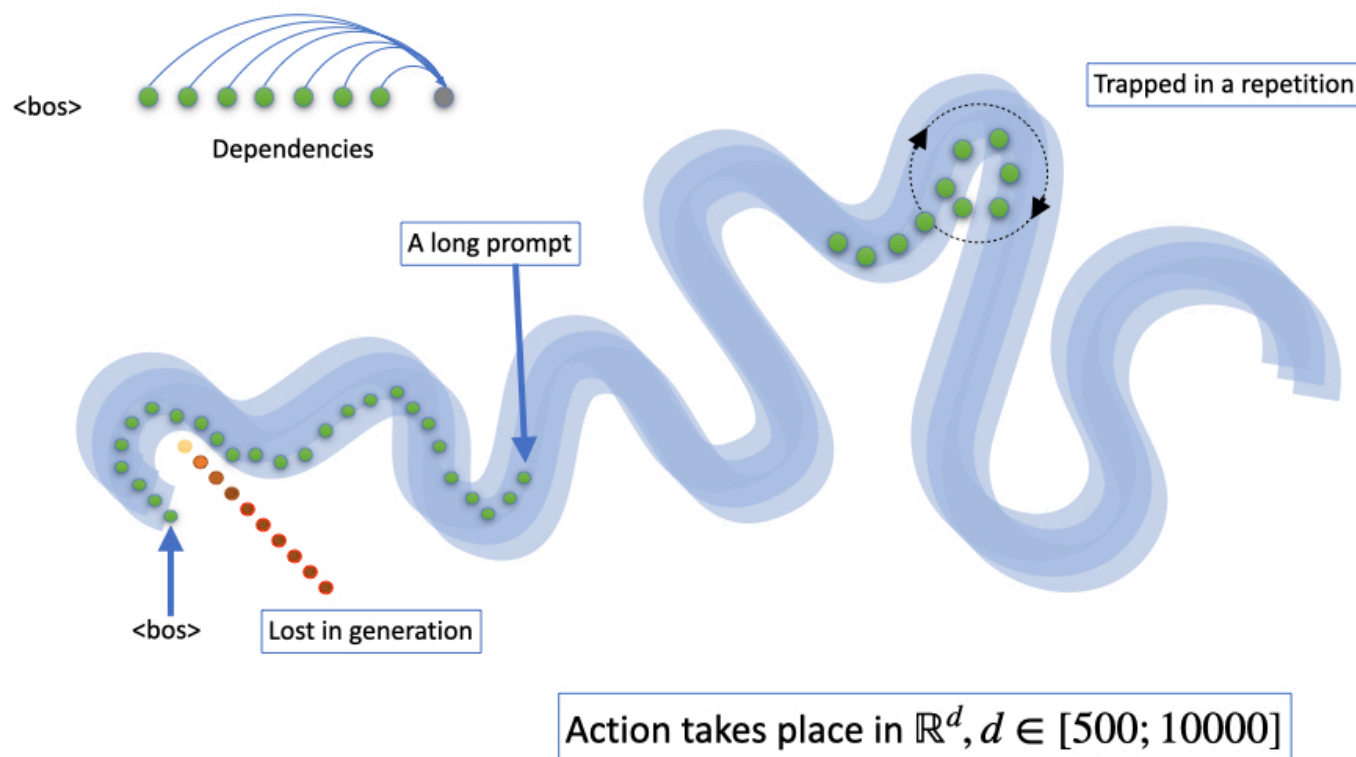
Generate diverse texts according to  $p_{\theta}(\mathbf{t}_{1:L})$

☞ Useful for data augmentation, text completion, creative writing...

## TRADE-OFF BETWEEN QUALITY AND DIVERSITY

☞ Decoding: high quality, low diversity

☞ Sampling: high diversity, lower quality



- High dimensional discrete space
  - ☞ Vocabulary size  $|\mathcal{T}|$  typically 30k-200k
  - ☞ Sentence of length  $L$ :  $|\mathcal{T}|^L$  possible sequences
- Long-range dependencies
  - ☞ Coherence, consistency over long texts

# TEXT GENERATION ► CHALLENGE 3: HOW TO EVALUATE GENERATED TEXTS?

(more on this in another lecture)

- N-gram overlap metrics (BLEU, ROUGE)
  - ☞ Need human reference
  - ☞ Poor correlation when model is strong
- Embedding-based metrics (BERTScore, MoverScore)
  - ☞ Better correlation but still needs reference
- Reference-less metrics (QuestEval, MAUVE)
  - ☞ Promising but still under active research
  - ☞ Rely on statistical models
- Human evaluation
  - ☞ Gold standard but costly and time-consuming

# **TEXT GENERATION**

## **GENERATION AS DECODING**

## Formalizing Text Generation as Search

(Un-)Conditional text generation is formalized as:

$$\mathbf{t}^* = \operatorname{argmax}_{\mathbf{t}_{P \dots L}} p_{\theta}(\mathbf{t}_{P+1 \dots L} | \mathbf{t}_{1:P})$$

☞ Finding  $L$  is part of the problem

Uses special tokens (*end of sequence*, e.g. )

Often bounded by a max length  $L_{max}$

☞ **Unconditional generation:** case  $P = 0$

## SIMPLEST DECODING STRATEGY

🔑 At each step, select the most probable token

$$\mathbf{t}_L = \operatorname{argmax}_{\mathbf{t}} p_{\theta}(\mathbf{t} | \mathbf{t}_{1..L-1})$$

### ALGORITHM

1. Start with prompt/prefix
2. Compute  $p_{\theta}(\cdot | \mathbf{t}_{1..L-1})$
3. Select  $\mathbf{t}_t = \operatorname{argmax}_{\mathbf{t}} p_{\theta}(\mathbf{t} | \mathbf{t}_{1..L-1})$
4. Repeat until EOS or max length

- ⊕ Fast and deterministic
- ⊕ Simple to implement
- ⊖ Myopic - no backtracking
- ⊖ Can lead to repetitive text
- ⊖ Locally optimal, not globally

 Lowerre, B.T. 1976. *The HARPY Speech Recognition System*.

## IDEA

- ☞ Maintain  $k$  best partial sequences (beams) at each step
- ☞ Explore multiple paths simultaneously

## WHY BEAM SEARCH?

- ☞ Greedy decoding: keeps only 1 hypothesis
- ☞ Exhaustive search:  $|\mathcal{T}|^L$  combinations (intractable)
- ☞ Beam search: middle ground with  $k$  hypotheses

### Beam width $k$

- ☞  $k = 1$ : equivalent to greedy
- ☞  $k = 5-10$ : typical for translation/summarization
- ☞ Larger  $k$  = more exploration but slower

## STEP-BY-STEP PROCESS

1. Initialize with  $K$  beams  $\mathbf{t}_{1:1}^{(k)}$  with log-probability  $s_{1:1}^{(k)} = \log p_{\theta}(\mathbf{t}_{1:1}^{(k)})$
2. For each step  $l$ :
  1. The candidates for the next step are all the sequences  $\mathbf{t}_{1:l} = \left( \mathbf{t}_{1:l-1}^{(k)}, t \right)_{t \in \mathcal{T}, k \in \{1, \dots, K\}}$
  2. Select those with the  $K$  highest scores (log-probability)

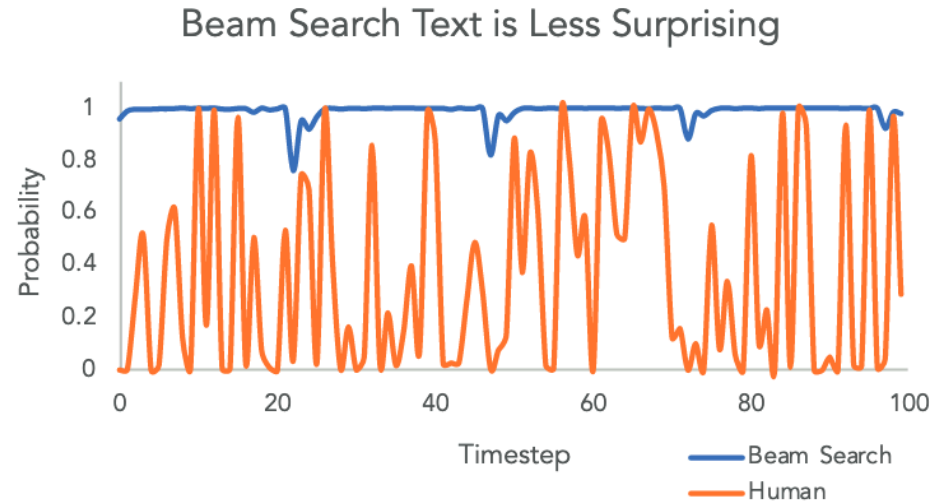
$$\log p_{\theta} \left( (\mathbf{t}_{1:l-1}^{(k)}, t) \right) = s_{1:l-1}^{(k)} + \log p_{\theta}(t | \mathbf{t}_{1:l-1}^{(k)})$$

3. Keep the  $K$  tokens with the highest  $s_{1:l}^{(k)} = s_{1:l-1}^{(k)} + \log p(\mathbf{t}_t^{(k)} | \mathbf{t}_{1:l-1}^{(k)})$
  4. Prune completed sequences (EOS)
3. Return highest-scoring complete sequence

☞ Note that many computations can be avoided ( $K$  most probable tokens per beam only)

# TEXT GENERATION ► BEAM SEARCH IS NOT SURPRISING

📖 Holtzman, A. et al. 2019. *The Curious Case of Neural Text Degeneration*.



## Beam Search

...to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and...

## Human

...which grant increased life span and three years warranty. The Antec HCG series consists of five models with capacities spanning from 400W to 900W. Here we should note that we have already tested the HCG-620 in a previous review and were quite satisfied With its performance. In today's review we will rigorously test the Antec HCG-520, which as its model number implies, has 520W capacity and contrary to Antec's strong beliefs in multi-rail PSUs is equipped...

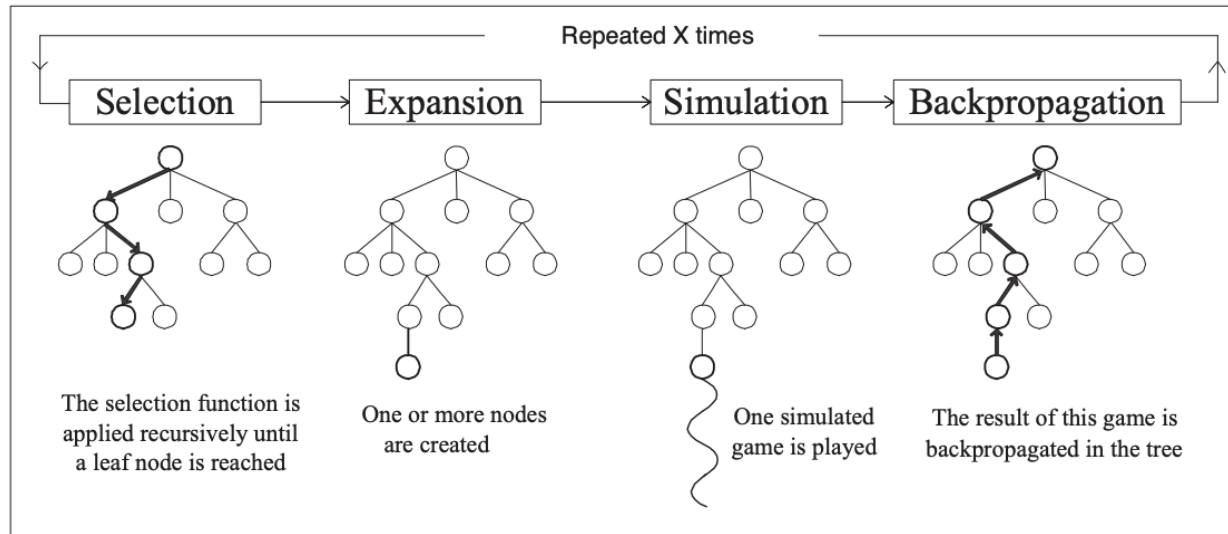


Figure 1: Outline of a Monte-Carlo Tree Search.

Chaslot, G. et al. 2008. Monte-Carlo Tree Search: A New Framework for Game AI.

## 💡 Mixes a min-max algorithm and Monte-Carlo

- 🔧 Usefull to play games
- 🔧 but also to generate sentences!
- 🔧 Often used with an auxilliary function

Scialom, T. et al. 2021. To Beam Or Not To Beam: That Is a Question of Cooperation for Language GANs.

# KNOWN ISSUES

## 1. Generic/Boring Outputs

- ☞ Favors safe, common sequences
- ☞ "I don't know" problem in QA
- ☞ **Solution: Add diversity penalty or use sampling**

## 2. Length Bias

- ☞ Without normalization, prefers shorter sequences (especially with high  $K$ )
- ☞ **Solution: Length normalization or length penalty**

## 3. Lack of Diversity

- ☞ All beams converge to similar sequences
- ☞ **Solution: Diverse beam search (group beams)**

## 4. Computational Cost

- ☞  $k \times$  slower than greedy
- ☞ Memory: need to store  $k$  sequences

# **TEXT GENERATION**

## **GENERATION AS SAMPLING**

## Generating text by sampling from a Language Model

Ancestral sampling principle:

1. Sample  $\mathbf{t}_1$  from  $P_{\theta}(\mathbf{t}_1)$
2. Sample  $\mathbf{t}_2$  from  $p_{\theta}(\mathbf{t}_2|\mathbf{t}_1)$
3. ...
4. Sample  $\mathbf{t}_L$  from  $p_{\theta}(\mathbf{t}_T|\mathbf{t}_{1:L-1})$

- ☑ Produces diverse texts
- ☒ but can produce low-probability (bad) texts

## Generating text by sampling from a Language Model

Ancestral sampling principle:

1. Sample  $\mathbf{t}_1$  from  $P_\theta(\mathbf{t}_1)$
2. Sample  $\mathbf{t}_2$  from  $p_\theta(\mathbf{t}_2|\mathbf{t}_1)$
3. ...
4. Sample  $\mathbf{t}_L$  from  $p_\theta(\mathbf{t}_L|\mathbf{t}_{1:L-1})$

- ☑ Produces diverse texts
- ☒ but can produce low-probability (bad) texts

☞ Top- $K$ , nucleus, typical, eta sampling *all* improve over basic sampling by restricting the set of tokens to sample  $\mathcal{T}^+ \subseteq \mathcal{T}$

📖 Fan, A. et al. 2018. Hierarchical Neural Story Generation. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

## IDEA

- 🔧 Sample only from the  $k$  most probable tokens at each step
- 🔧 Prevents unlikely (nonsensical) tokens

## ALGORITHM

1. Compute  $p_{\theta}(\mathbf{t}_t | \mathbf{t}_{1:l-1})$
2. Keep top- $k$  tokens  $\mathcal{T}^+$
3. Renormalize:  $p'(t) = \frac{p(t | \mathbf{t}_{1:l-1})}{\sum_{t' \in \mathcal{T}^+} p(t' | \mathbf{t}_{1:l-1})}$
4. Sample from  $p'(t)$

- ✅ Prevents very unlikely tokens
- ✅ Simple and fast
- ❌ Fixed  $k$  doesn't adapt to distribution
- ❌ May be too restrictive or too loose

# TEXT GENERATION ► NUCLEUS (TOP-P) SAMPLING

 Holtzman, A. et al. 2019. *The Curious Case of Neural Text Degeneration*.

## IDEA

-  Dynamically select smallest set with cumulative probability  $\geq p$
-  Adapts to model's confidence

## ALGORITHM

1. Find the smallest  $\mathcal{T}^{(p)} \subseteq \mathcal{T}$  where  $\sum_{v \in \mathcal{T}^{(p)}} p_{\theta}(t | \mathbf{t}_{1:L}) \geq p$
2. Renormalize and sample from  $\mathcal{T}^{(p)}$

### Adaptive behavior

-  Confident prediction  $\rightarrow$  small nucleus (few tokens)
-  Uncertain prediction  $\rightarrow$  large nucleus (many tokens)
-  Typical:  $p = 0.9$  or  $p = 0.95$

-  Adapts to distribution shape
-  Better than fixed top-k
-  Still may include unlikely tokens if distribution is flat
-  May exclude likely tokens if distribution is sharp

 Meister, C. et al. 2023. *Locally Typical Sampling*.

## MOTIVATION

- ☞ High probability  $\neq$  typical (psycholinguistics studies)
- ☞ Information content matters: surprisal  $= -\log p(x)$

## KEY INSIGHT

- ☞ Sample tokens with information close to expected
- ☞ Avoid both too surprising and too boring

## ALGORITHM

1. Compute entropy:  $H = -\sum_x p(x) \log p(x)$
2. For each token: deviation  $= |\log p(x) - H|$
3. Keep tokens with small deviation (within  $\tau$ )
4. Sample from this set

-  More human-like text
-  Balances predictability and surprise

 *Truncation Sampling as Language Model Desmoothing: 2022. <http://arxiv.org/abs/2210.15191>.*

## IDEA

-  Entropy-aware adaptive threshold
-  Adjust cutoff based on distribution entropy

## METHOD

Sample from

$$\mathcal{T}^+ = \left\{ t : p(t) > \eta \cdot \max_{t'} p(t') \right\} \text{ with } \eta = \min(\epsilon, \alpha \exp -H(t))$$

where  $H$  is the entropy of  $p(\cdot)$



### Adaptive threshold

-  Low entropy (confident)  $\rightarrow$  stricter  $\eta$  (high threshold)
-  High entropy (uncertain)  $\rightarrow$  relaxed  $\eta$  (low threshold)

-  Adapts to model confidence
-  Good quality-diversity balance

## Consistent model

📖 Welleck, S. et al. 2020. Consistency of a Recurrent Language Model With Respect to Incomplete Decoding. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

A **consistent model** is such that  $p_{\theta}(L = \infty) = 0$

## Consistent model

Welleck, S. et al. 2020. Consistency of a Recurrent Language Model With Respect to Incomplete Decoding. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP).

A **consistent model** is such that  $p_{\theta}(L = \infty) = 0$

## ⚠ Inconsistency of standard decoding

Many decoding and sampling methods (greedy, beam search, top- $k$ , typical,  $\eta$ , etc.) *may* produce infinite sequences with non-zero probability even if the model is consistent

## Consistent model

 Welleck, S. et al. 2020. Consistency of a Recurrent Language Model With Respect to Incomplete Decoding. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP).

A **consistent model** is such that  $p_{\theta}(L = \infty) = 0$

## ⚠ Inconsistency of standard decoding

Many decoding and sampling methods (greedy, beam search, top- $k$ , typical,  $\eta$ , etc.) *may* produce infinite sequences with non-zero probability even if the model is consistent

## 💡 Solution: enforce end-of-sequence token generation

- Include the EOS token in the candidate set at each step
  - Modify decoding/sampling to **always** so that the probability of generation of the EOS token increases at each step
- Sufficient condition:  $\exists \epsilon, p(eos) > \epsilon$

# **TEXT GENERATION**

## **CONSTRAINED GENERATION**

## WHY CONSTRAIN GENERATION?

- 🔑 Control attributes of generated text without retraining
- Control sentiment (positive/negative)
  - Control topic (science, politics, sports)
  - Control style (formal/informal)
  - Satisfy hard constraints (keywords, format)
  - Avoid toxicity or bias

## WHY CONSTRAIN GENERATION?

- 🔑 Control attributes of generated text without retraining
- Control sentiment (positive/negative)
- Control topic (science, politics, sports)
- Control style (formal/informal)
- Satisfy hard constraints (keywords, format)
- Avoid toxicity or bias

## TYPES OF CONSTRAINTS

### Hard Constraints

Enforce specific tokens or patterns (*practical*)

Can be tricky

📖 *NeuroLogic Decoding: (Un)Supervised Neural Text Generation with Predicate Logic Constraints: 2021.* <http://arxiv.org/abs/2010.12884>.

### Soft Constraints

Encourage certain attributes (sentiment, topic)

Methods: PPLM, FUDGE, GeDi

📖 *Krause, B. et al. 2020. GeDi: Generative Discriminator Guided Sequence Generation.*

📖 *Plug and Play Language Models: A Simple Approach to Controlled Text Generation: 2020.* <http://arxiv.org/abs/1912.02164>.

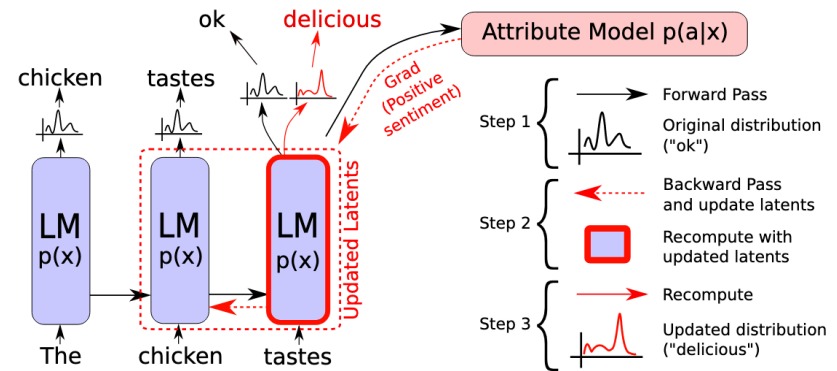
📖 *Yang, K. and Klein, D. 2021. FUDGE: Controlled Text Generation With Future Discriminators. Proceedings of the 2021 Conference of the North American Chapter*

# TEXT GENERATION ► PLUG AND PLAY LANGUAGE MODELS (PPLM)

📖 *Plug and Play Language Models: A Simple Approach to Controlled Text Generation: 2020. <http://arxiv.org/abs/1912.02164>.*

## KEY IDEA

- 🔧 Steer generation by modifying hidden states at each step
- 🔧 No retraining needed - works with any pre-trained LM!



## HOW IT WORKS

1. Generate token representation  $\mathbf{h}_i$  from LM
2. Compute gradient of attribute model  $\log p(a_i | \mathbf{t}_{1:i})$  w.r.t.  $\mathbf{h}_i$
3. Update  $\mathbf{h}_i$  in direction that increases desired attribute
4. Generate token from modified  $\mathbf{h}_i$
5. Repeat for each token

- ⊕ No model retraining required
- ⊕ Can combine multiple attributes
- ⊖ Slower generation (gradient computation)
- ⊖ May reduce fluency

 *Plug and Play Language Models: A Simple Approach to Controlled Text Generation: 2020. <http://arxiv.org/abs/1912.02164>.*

## OBJECTIVE

Maximize both language model likelihood and attribute model score:

$$p(\mathbf{t}|a) \propto p(\mathbf{t}) \cdot p(a|\mathbf{t})$$

 where  $a$  is the desired attribute (e.g., positive sentiment)

## GRADIENT UPDATE

At each step, update hidden state:

$$\mathbf{h}_i \leftarrow \mathbf{h}_i + \alpha \nabla_{\mathbf{h}_i} \log p(a|\mathbf{h}_i)$$

  $\alpha$  controls strength of attribute

## Attribute Models

-  Bag of Words: encourage specific keywords
-  Discriminator: neural classifier for sentiment/topic
-  Multiple attributes can be combined

## KEY IDEA

☞ Use auxiliary models to guide generation based on desired attributes

### GeDI (GENERATIVE DISCRIMINATOR)

📖 Krause, B. et al. 2020. *GeDi: Generative Discriminator Guided Sequence Generation*.

☞ **Generative approach:** Uses small control-code LMs  $p(\mathbf{t}|a)$

☞ Trained on class-specific data with control codes

☞ At generation: compare likelihoods across classes

$$p(a|\mathbf{t}) \propto \frac{p(\mathbf{t}|a)}{p(\mathbf{t})}$$

⊕ Fast inference (no gradients)

⊕ Strong attribute control

⊖ Requires training class-conditional LMs

### FUDGE (FUTURE DISCRIMINATOR)

📖 Yang, K. and Klein, D. 2021. *FUDGE: Controlled Text Generation With Future Discriminators*. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

☞ Predicts attribute from *future* tokens (not just past)

☞ At generation: scores token by future attribute prediction

$$p(\mathbf{t}_i|\mathbf{t}_{1:i}, a) \propto p(\mathbf{t}_i|\mathbf{t}_{1:i-1}) \cdot p(a|prefix = \mathbf{t}_{1:i})$$

⊕ More fluent than PPLM (Bayesian guidance)

⊕ Can use any base LM (no retraining)

⊖ Requires training discriminator model

⊖ Slower than GeDi (Bayesian inference)

## EXACT CONSTRAINT SATISFACTION

- ☞ Enforce specific tokens, phrases, or logical constraints
- ☞ Useful for controllable generation with guarantees

## CONSTRAINED SEARCH

- ☞ Modify search to only keep sequences satisfying constraints
- ☞ Constraints: required keywords, forbidden tokens, formats

### Use cases

- ☞ "Generate review containing 'excellent' and 'innovative'"
- ☞ "Generate text avoiding toxic words"
- ☞ "Follow specific format (e.g., JSON structure)"
- ☞ Uses a Finite State Automata

- ⊕ Guarantees constraint satisfaction (somehow)
- ⊕ No extra models needed
- ⊕ Simple to implement
- ⊖ Limited to exact token matching
- ⊖ Can reduce fluency if constraints too strict
- ⊖ Exponential complexity for complex constraints

## NEUROLOGIC DECODING

 *NeuroLogic Decoding: (Un)Supervised Neural Text Generation with Predicate Logic Constraints: 2021. <http://arxiv.org/abs/2010.12884>.*

-  Combines beam search with **constraint satisfaction**
-  Supports complex logical constraints (AND, OR, NOT)
-  Uses satisfiability solver to prune invalid beams
-  Modified beam search (scores for *how satisfiable* is the output so far)

### Advanced constraints

-  "Include 'climate' AND ('warming' OR 'change')"
-  "Mention 'Paris' but NOT in context of 'fashion'"
-  Lexical constraints with logical operators

-  Flexible constraint specification
-  Handles complex logical formulas
-  Better quality than greedy constraint matching
-  More complex implementation
-  Higher computational cost

# TEXT GENERATION

## CONCLUSION

# KEY CONCEPTS COVERED

## DECODING METHODS

- Greedy decoding: fast but myopic
- Beam search: explores multiple paths
- MCTS: combines search and sampling

## SAMPLING METHODS

- Ancestral: diverse but risky
- Top-K: fixed threshold
- Nucleus (Top-p): adaptive probability
- Typical & Eta: entropy-aware

## CONSTRAINED GENERATION

- **Soft:** PPLM, GeDi, FUDGE
  - ☞ Control attributes via guidance
- **Hard:** Constrained beam search, NeuroLogic
  - ☞ Enforce exact constraints

## QUALITY VS. DIVERSITY TRADE-OFF

- ☞ Decoding: high quality, low diversity
- ☞ Sampling: high diversity, variable quality
- ☞ *The eternal dilemma*

## ⚠ Why is text generation still challenging?

### 1. Combinatorial Explosion

- ☞ Search space:  $|\mathcal{T}|^L$  (intractable)

### 2. Quality-Diversity Trade-off

- ☞ Decoding: high-prob, low-diversity
- ☞ Sampling: diverse, lower quality

### 3. Evaluation is Difficult

- ☞ Metrics correlate poorly with humans
- ☞ No universal metric for all aspects

### 4. Long-Range Dependencies

- ☞ Maintaining consistency is hard
- ☞ Exposure bias during training

### 5. Controllability vs. Fluency

- ☞ Strong constraints → reduced fluency
- ☞ Weak constraints → poor control

### 6. Task-Specific Requirements

- ☞ Translation: beam search
- ☞ Creative writing: sampling
- ☞ No one-size-fits-all solution

## ⚠ Why is text generation still challenging?

### 1. Combinatorial Explosion

- ☞ Search space:  $|\mathcal{T}|^L$  (intractable)

### 2. Quality-Diversity Trade-off

- ☞ Decoding: high-prob, low-diversity
- ☞ Sampling: diverse, lower quality

### 3. Evaluation is Difficult

- ☞ Metrics correlate poorly with humans
- ☞ No universal metric for all aspects

### 4. Long-Range Dependencies

- ☞ Maintaining consistency is hard
- ☞ Exposure bias during training

### 5. Controllability vs. Fluency

- ☞ Strong constraints → reduced fluency
- ☞ Weak constraints → poor control

### 6. Task-Specific Requirements

- ☞ Translation: beam search
- ☞ Creative writing: sampling
- ☞ No one-size-fits-all solution

**Text generation remains one of the most exciting and challenging areas in NLP!**